Amazon SageMaker AI 调研报告

1. 产品定位与入口

SageMaker AI 的功能并不局限于训练作业，而是覆盖：

环境配置；SageMaker Studio / Notebook / IDE；Training & tuning jobs；JumpStart model hub；HyperPod；Deployments & inference；Model governance；Data preparation 等多个功能。

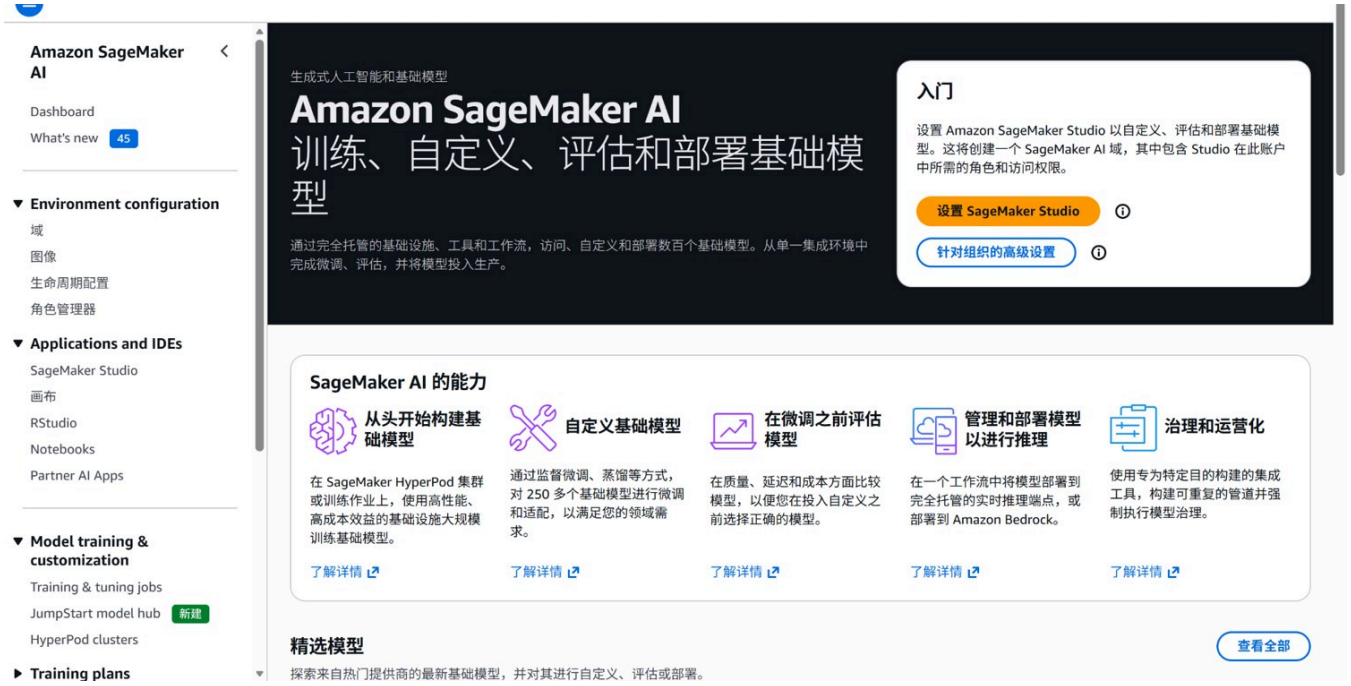


图 1：Amazon SageMaker AI 控制台主页

调研结论

SageMaker AI 的产品逻辑是围绕“模型生命周期”组织能力，而不是只提供一个“训练”按钮。对本项目而言，也更适合把模型、数据集、训练任务、模型产物、部署任务分别建模，而不是全部塞进单一训练页面。

---

2. JumpStart Model Hub：模型选择与模型详情

2.1 模型中心

进入 JumpStart Model Hub 后，可以通过搜索和筛选寻找模型。实际页面提供了包括输入模态、输出模态、发布者、使用场景、模型访问权限、许可证、模型大小等筛选条件。

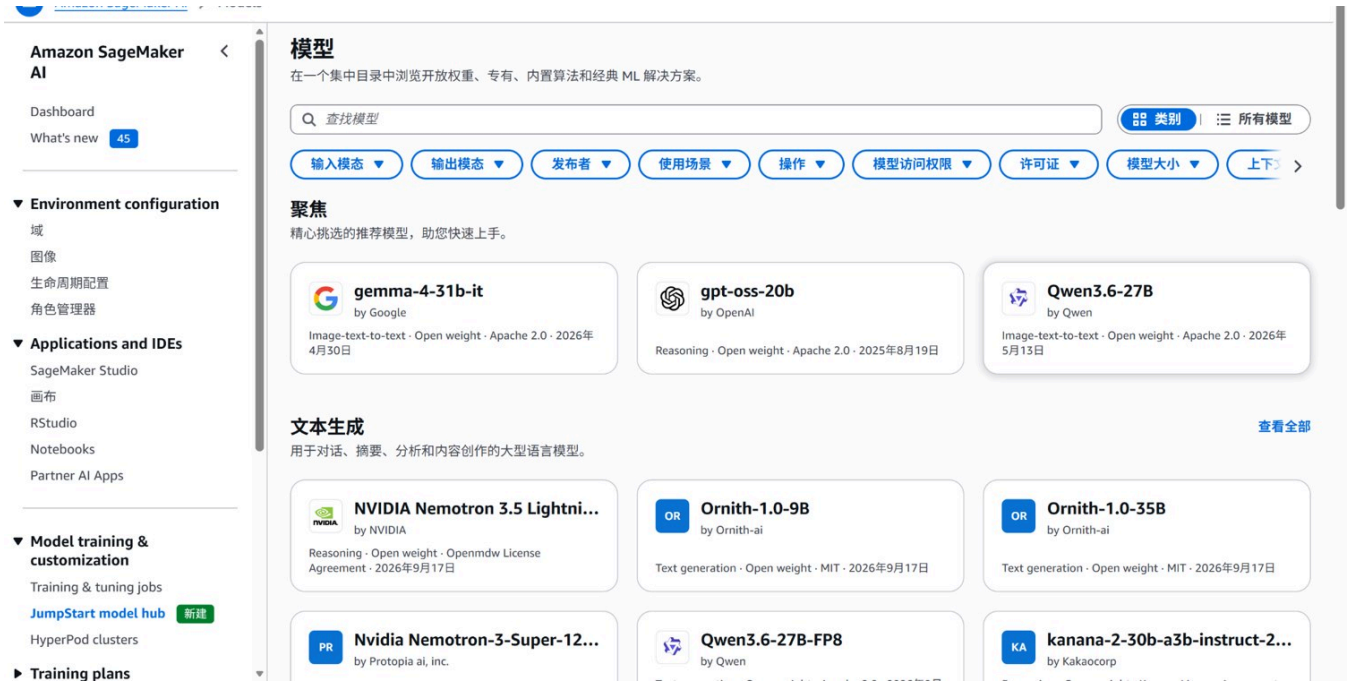


图 2：JumpStart Model Hub

模型以卡片形式展示，用户在进入详情页前就能够快速看到模型名称、发布者、模型用途、许可类型等基础信息。

2.2 模型详情

本次实际查看了 Qwen 系列模型详情。详情页展示：

模型提供商；许可证；模型大小；输入模态；输出模态；框架；是否支持自定义；可用的模型自定义方式；进入 Studio、JupyterLab、Code Editor 等操作入口。

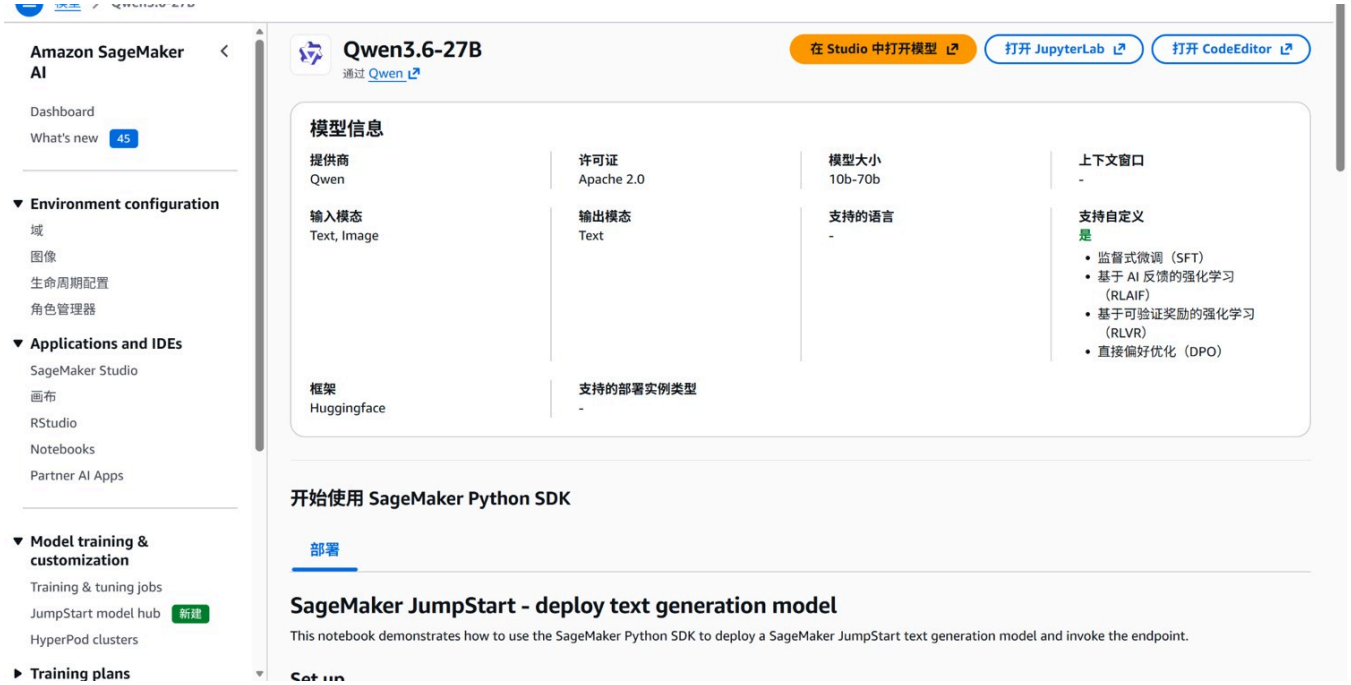


图 3：模型详情页

可以学习

3. Training & Tuning Jobs：训练任务管理

进入 **Training & tuning jobs**。

训练作业列表页用于：查看历史训练任务；查看任务状态；搜索训练任务；创建 Training Job。



图 4：Training Jobs 列表

“将“一次训练”作为独立对象管理。

---

4. 创建 Training Job

实际打开“创建训练作业”后，一次训练任务包含多个配置维度。

4.1 基础信息与算法来源

首先需要配置：

- 作业名称；
- IAM 执行角色；
- 算法来源。

算法来源包括：

- SageMaker 内置算法；
- 用户自己的算法资源；
- ECR 中的自定义训练容器；
- AWS Marketplace 算法。

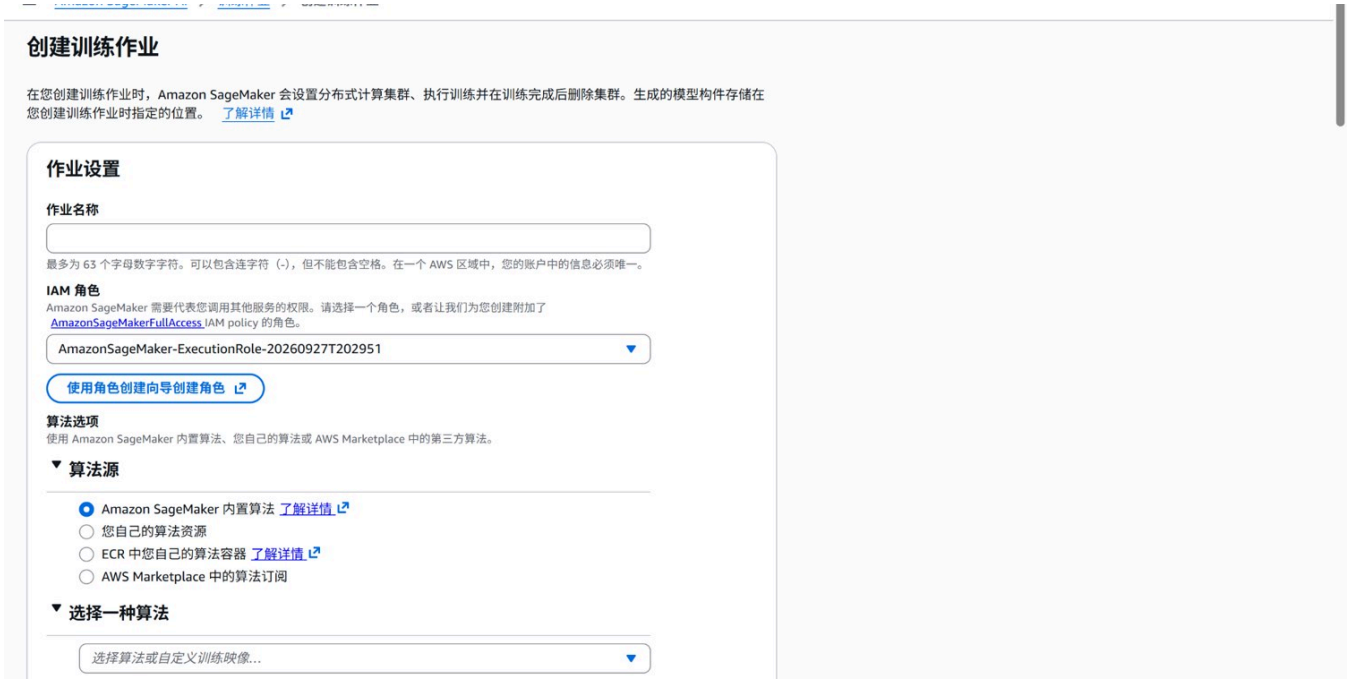


图 5：Training Job 基础配置

对本项目的启发

训练平台可以保留两种模式：

快速模式

```
选择模型
→ 选择数据集
→ 选择 GPU
→ 配核心参数
→ 训练
```

高级模式

```
自定义镜像
→ 环境变量
→ 分布式设置
→ 网络
→ 存储
→ Checkpoint
→ 高级资源策略
```

这样既能让普通用户快速上手，也能满足高级用户自定义训练环境的需求。

5. 计算资源配置

SageMaker 用“实例类型”将底层 CPU、内存、GPU 等硬件封装成标准资源规格，用户通过实例类型、实例数量、容量来源和存储大小描述一次训练所需的资源；平台随后结合账号配额和实际容量决定是否真正为 Training Job 分配计算资源

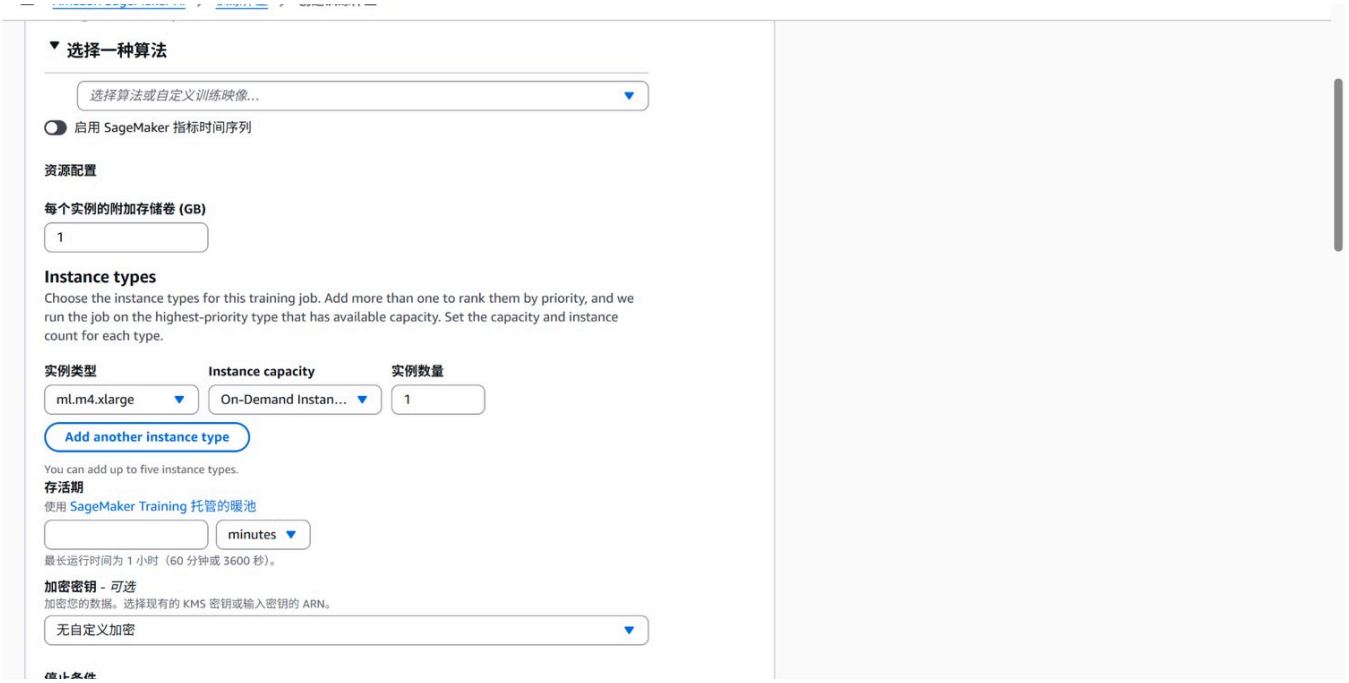


图 6：训练资源配置

实例类型直接决定训练使用的 CPU/GPU 资源。实例数量可以配置，平台具备多实例训练的基础。

5.1多候选实例

页面支持添加多个 Instance Type。

```
首选 GPU
   ↓ 无可用容量
备选 GPU
   ↓
可用则运行
```

- 首选资源；
- 备选资源；
- 无资源时进入队列；
- 展示预计等待时间。

5.2 最大运行时间

平台允许设置 Training Job 的最大运行时长。

最大运行时间；

最大 GPU 小时；

超过上限自动终止；

提交前费用/资源消耗提示。

6. 超参数设计

选择具体算法后，SageMaker 会自动展示该算法对应的超参数。

本次为了尝试低成本完整训练，选择了 **Tabular - XGBoost**，并将任务设为二分类。

主要配置如下：

```
objective = binary:logistic
num_round = 20
max_depth = 3
eta = 0.2
eval_metric = auc
```

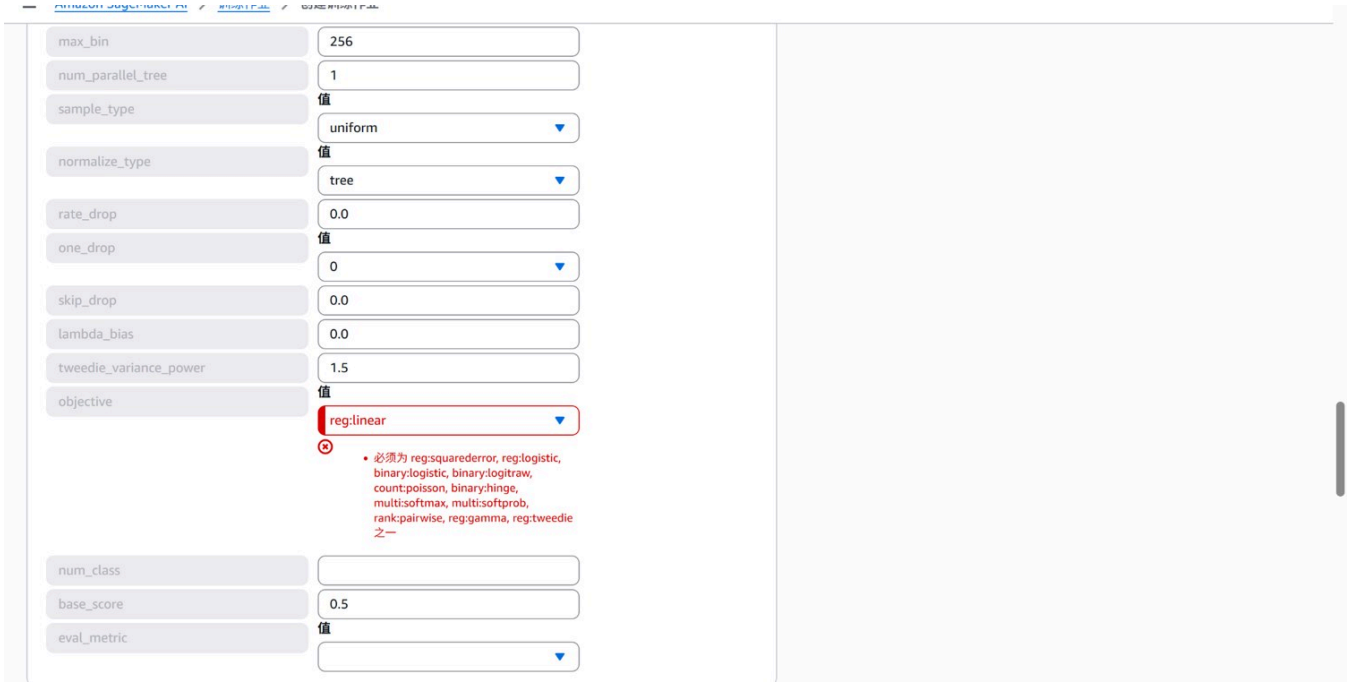


图 7：XGBoost 超参数

在实际填写过程中，当 `objective` 使用无效值时，页面会直接标红并给出允许值。

SageMaker 的超参数设计会根据算法动态展示参数；提交前进行合法性校验。

不应让用户面对一个通用的“键值对参数编辑器”作为唯一入口。建议：

普通用户使用模板化表单；参数显示中文说明；给出推荐值；设置合法范围；提交前进行校验；高级用户可切换到自由参数模式。方便上手

---

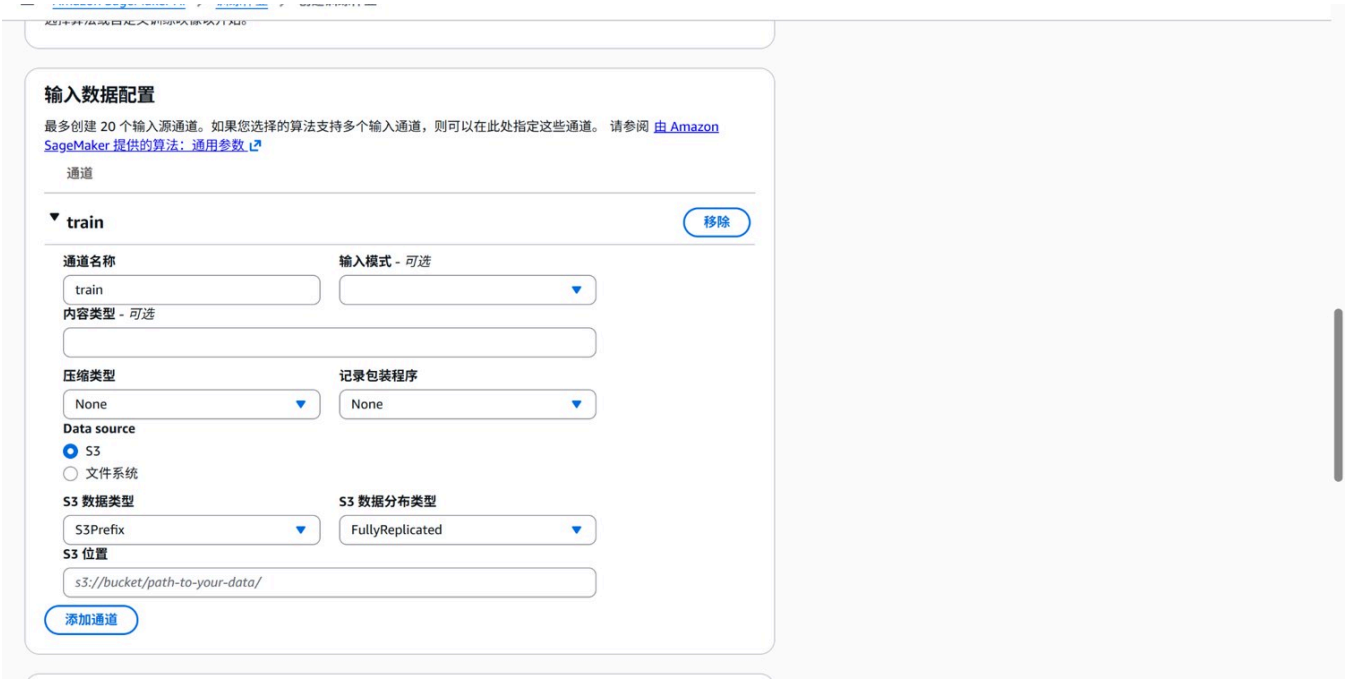
7. 输入数据：Channel + S3

SageMaker 将训练数据抽象成 **Input Channel**。

S3存数据，以url的形式进行输入

数据集需要提前上传到 S3，然后 Training Job 只保存数据位置。

```
数据先进入对象存储
        ↓
Training Job 引用数据 URI
        ↓
训练节点读取数据
```



7.1 Checkpoint

Checkpoint 用于保存训练的中间状态

减少长时间训练因为节点故障或被抢占而重新开始造成的资源浪费。

7.2 输出数据

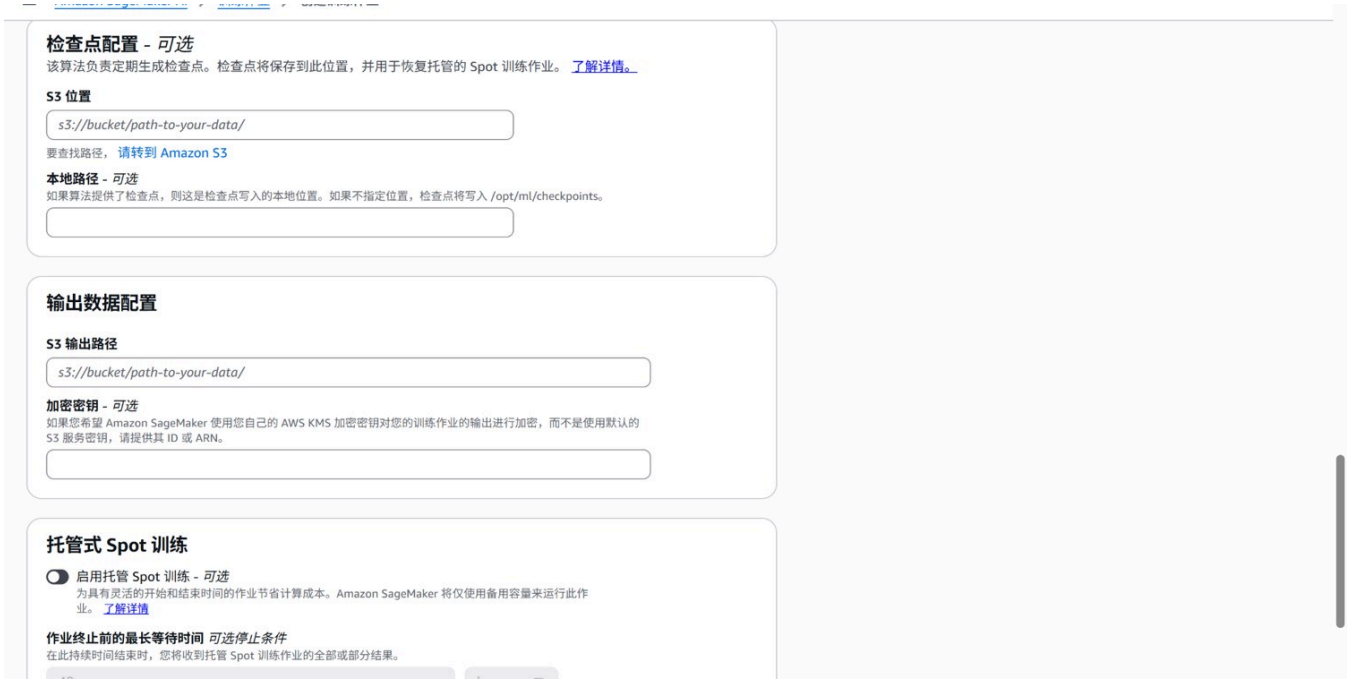
训练完成后的模型产物写到指定位置。

可抽象为：

```
Dataset
   ↓
Training Job
   ↓
Model Artifact
```

8. Managed Spot Training

SageMaker 还支持 Managed Spot Training。



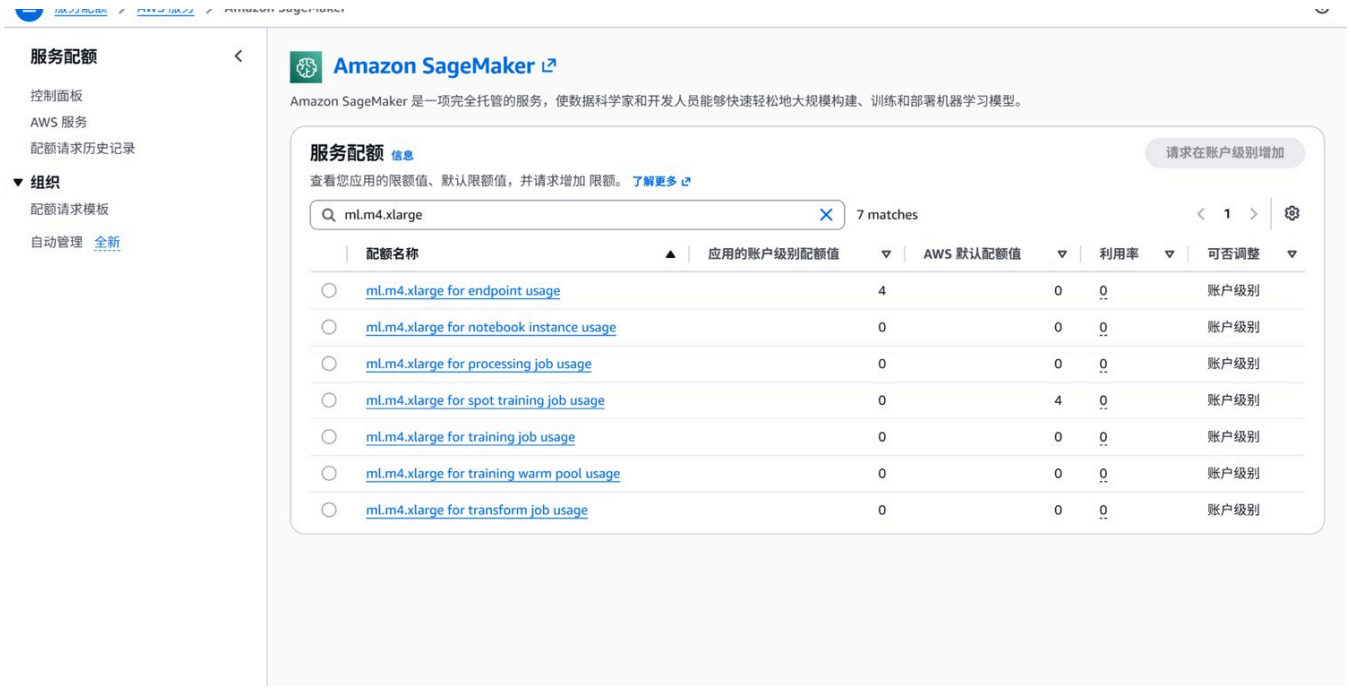
Managed Spot Training 利用可被回收的空闲计算资源降低训练成本。任务可能因资源回收而中断，因此通常需要与 Checkpoint 配合，在重新获得资源后从最近保存的状态继续训练。低优先级、可抢占式 GPU 调度策略。

9. Service Quotas：资源额度管理

控制台Service Quotas 可以查看相应的资源配额

相同实例在不同用途下额度不同

同一种实例按照用途分为多个usage



SageMaker 功能非常完整，但它暴露了很多 AWS 基础设施概念，例如：

IAM；S3；ECR；VPC；KMS；Service Quotas；Instance Type。

对于学生或普通算法用户，这些概念有较高学习门槛。

一些功能比较分散，资源额度没有在训练页面直接展示

可以采用“底层完整、前端简化”的方式。

将一些内容放入扩展设置

1. **Training Job 作为核心业务对象；**
2. **模型、数据集和训练任务相互独立；**
3. **根据模型/算法动态生成参数；**
4. **Checkpoint 支持长任务容错；**
5. **普通资源与可抢占资源采用不同调度策略；**
6. **失败原因应明确指向具体资源、参数或权限问题；**
7. **训练产物应自动进入模型资产管理流程。**